

Module 3 Introduction to OOPS Programming

THEORY

1. What are the key differences between Procedural Programming and Object-Oriented Programming (OOP)?

❖ Key Differences Between Procedural Programming (POP) and Object-Oriented Programming (OOP)

Feature	Procedural Programming (POP)	Object-Oriented Programming (OOP)
Approach	Follows a top-down approach	Follows a bottom-up approach
Focus	Focuses on functions (procedures)	Focuses on objects and classes
Data Handling	Data is exposed and can be accessed freely by functions	Data is hidden using encapsulation
Security	Less secure (global data is common)	More secure (data is protected inside objects)
Reusability	Limited reusability	High reusability through classes & inheritance
Examples	C, Pascal	C++, Java, Python

2. List and explain the main advantages of OOP over POP.

❖ Main Advantages of OOP Over POP

1. Encapsulation

Data and functions are bundled together inside objects.

Protects data from unauthorized access.

2. Abstraction

Shows only essential features and hides internal details.

Makes complex systems easier to understand.

3. Inheritance

Allows one class to acquire properties of another.

Promotes code reusability and reduces redundancy.

4. Polymorphism

Same function or operator can behave differently in different contexts.

Increases flexibility in code.

5. Modularity

Large programs can be broken into smaller, manageable modules (classes).

Easier to debug, maintain, and update.

3. Explain the steps involved in setting up a C++ development environment.

Steps Involved in Setting Up a C++ Development Environment

Step 1: Install a C++ Compiler

Use one of the following:

GCC/G++ (Windows via MinGW, Linux, macOS)

Clang

MSVC (Microsoft Visual C++)

Step 2: Install an IDE or Code Editor

Popular choices:

Visual Studio Code

Code::Blocks

Dev-C++

CLion

Visual Studio (Windows-only)

Step 3: Configure the Compiler in the Editor

VS Code requires the **C/C++ extension**.

Link the installed compiler (e.g., MinGW) to the PATH.

Step 4: Write Your First C++ File

Create a file with `.cpp` extension

Example: `hello.cpp`

Step 5: Compile the Program

Using terminal/command prompt:

```
g++ hello.cpp -o hello
```

Step 6: Run the Program

```
./hello      # on Linux or Mac
```

```
hello.exe    # on Windows
```

4. What are the main input/output operations in C++? Provide examples.

A. Output Operation: `cout`

Used to display data on the screen.

Example:

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    cout << "Hello, World!";
```

```
    return 0;
```

```
}
```

B. Input Operation: `cin`

Used to take input from the user.

Example:

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int age;
```

```
    cout << "Enter your age: ";
```

```
    cin >> age;
```

```
    cout << "Your age is: " << age;
```

```
    return 0;
```

```
}
```

C. Other I/O Objects

Object Purpose

cout Output to console

cin Input from user

cerr Display error messages

clog Log messages

5.What are the different data types available in C++? Explain with examples

❖ C++ data types are broadly classified into 3 categories:

1.Built-in (Primitive) Data Types

2.Derived Data Types

3.User-defined Data Types

1. Built-in (Primitive) Data Types

These are the basic data types provided by the language.

Data Type	Description	Example
int	Stores integers (whole numbers)	<code>int age = 20;</code>
float	Stores decimal numbers (single precision)	<code>float price = 10.99;</code>
double	Stores decimal numbers (double precision)	<code>double pi = 3.14159;</code>
char	Stores a single character	<code>char grade = 'A';</code>
bool	Stores true/false values	<code>bool isValid = true;</code>
void	No value; used in functions	<code>void display();</code>
wchar_t	Stores wide characters	<code>wchar_t w = L'Ω'; (Greek Omega)</code>

Example:

```
int num = 10;
```

```
float marks = 88.5f;
```

```
double salary = 55000.75;
```

```
char letter = 'B';
```

```
bool passed = true;
```

2.Derived Data Types

These are built using primitive types.

Data Type	Description	Example
Array	Collection of similar type elements	<code>int arr[5];</code>
Pointer	Stores memory address	<code>int *ptr;</code>
Function	A block of reusable code	<code>int add(int a, int b);</code>
Reference	Alias for another variable	<code>int &ref = x;</code>

Example:

```
int numbers[3] = {1, 2, 3};
```

```
int x = 5;
```

```
int *ptr = &x; // pointer
```

```
int &ref = x; // reference
```

3.User-defined Data Types

Created by programmers to build complex structures.

Data Type	Description	Example
struct	Group of different data types	<code>struct Student { ... };</code>
class	Blueprint for objects (OOP)	<code>class Car { ... };</code>
enum	Set of named integer constants	<code>enum Day { Sun, Mon };</code>
typedef	Creates alias for a type	<code>typedef int number;</code>
union	Shares memory for different variables	<code>union Data { ... };</code>

Example:

```
struct Student {  
    int roll;  
    float marks;  
};  
  
enum Day { Monday, Tuesday, Wednesday };
```

6. Explain the difference between implicit and explicit type conversion in C++.

❖ Implicit vs Explicit Type Conversion

1. Implicit Type Conversion (Automatic Conversion)

Compiler **automatically** converts one data type to another.

Safe conversions only.

Example:

```
int a = 10;  
  
double b = a;    // int → double (automatic)
```

2. Explicit Type Conversion (Manual Conversion)

Programmer **forcefully** converts one type to another.

Can cause data loss.

Example:

```
double x = 9.8;  
  
int y = (int)x;    // double → int (manual)
```

Difference

Implicit

Explicit

Automatic

Manual

Done by compiler Done by programmer

Safe conversions Risky conversions

No cast needed Cast required

7. What are the different types of operators in C++? Provide examples of each

❖ Type	Examples
Arithmetic	+, -, *, /, %
Relational	==, !=, >, <
Logical	&&, , !
Assignment	=, +=, -=
Increment/Decrement	++, --
Bitwise	&,
Conditional	?:
Special	sizeof, &, *

1.Arithmetic Operators

Used for mathematical calculations.

- **Examples:**

+	Addition	(a + b)
-	Subtraction	(a - b)
*	Multiplication	(a * b)
/	Division	(a / b)
%	Modulus	(a % b)

2. Relational (Comparison) Operators

Used to compare values, result is **true/false**.

Examples:

==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater or equal
<=	Less or equal

3. Logical Operators

Used to combine conditions.

Examples:

&&	Logical AND
	Logical OR
!	Logical NOT

4. Assignment Operators

Used to assign values to variables.

Examples:

```
=      a = 5

+=     a += 3    // a = a + 3

-=     a -= 2

*=     a *= 4

/=     a /= 2

%=     a %= 3
```

5. Increment and Decrement Operators

Increase or decrease value by 1.

Examples:

```
++a    Pre-increment
a++    Post-increment
--a    Pre-decrement
a--    Post-decrement
```

6 Bitwise Operators

Operate on bits (0s and 1s).

Examples:

```
&      Bitwise AND
|      Bitwise OR
^      Bitwise XOR
~      Bitwise NOT
<<     Left shift
>>     Right shift
```

7. Conditional (Ternary) Operator

Short form of if-else.

Example:

```
condition ? value1 : value2;
```

```
int result = (a > b) ? a : b;
```

8. Special Operators

Includes various important operators:

1. sizeof

```
int a;
```

```
sizeof(a) ;
```

2. & (address-of operator)

```
int x;
```

```
&x;
```

3. * (pointer dereference)

```
int *p;
```

```
*p;
```

4. typeid (used in RTTI)

```
typeid(a).name();
```

8. Explain the purpose and use of constants and literals in C++

1.Constants

Definition:

Constants are values in C++ that **cannot be changed** during program execution.

Purpose:

To store **fixed values**

To prevent accidental modification

To make programs safer and more readable

Types of Constants:

const keyword

#define macro

Examples:

Using `const`

```
const int MAX = 100;
```

Using `#define`

```
#define PI 3.14
```

You **cannot** change these values later:

```
MAX = 50; // ❌ Error
```

```
PI = 3.5; // ❌ Error
```

2. Literals

Definition:

Literals are **fixed values** written directly in the code.

Example:

10, 3.14, 'A', "Hello", true

Types of Literals:

Type	Example
Integer literal	10, -5, 200
Floating literal	3.14, 5.0

Type	Example
Character literal	'a', 'Z'
String literal	"Hello"
Boolean literal	true, false

□ Purpose of Literals

Provide constant values to variables

Represent fixed data in the program

Used directly in expressions

Example:

```
int age = 18;      // 18 is a literal
```

```
float pi = 3.14;   // 3.14 is a literal
```

```
char ch = 'A';     // 'A' is a literal
```

9. What are conditional statements in C++? Explain the if-else and switch statements

. Conditional Statements in C++ (if-else & switch)

Conditional Statements

Used to make decisions based on conditions (true/false).

▣ if-else Statement

Used when you want to execute different code based on a condition.

Syntax:

```
if (condition) {
```

```
        // runs if condition is true
    }

    else {

        // runs if condition is false
    }
```

Example:

```
int marks = 60;

if (marks >= 40)

    cout << "Pass";

else

    cout << "Fail";
```

switch Statement

Used when you want to choose one option from many fixed choices.

Syntax:

```
switch (value) {

    case 1: ... break;

    case 2: ... break;

    default: ...

}
```

Example:

```
int day = 2;

switch (day) {

    case 1: cout << "Monday"; break;
```

```

    case 2: cout << "Tuesday"; break;

    default: cout << "Other day";

}

```

10. What is the difference between for, while, and do-while loops in C++?

❖ Difference Between for, while, and do-while Loops

Loop Type	When It Runs	Condition Check	Use Case
for	Known number of iterations	Before loop	Best for counting loops
while	Unknown iterations	Before loop	Loop continues until condition becomes false
do-while	Runs at least once	After loop	When you want loop to run at least once

Examples:

for

```

for (int i = 1; i <= 5; i++)
    cout << i;

```

while

```

int i = 1;

while (i <= 5) {
    cout << i;

    i++;
}

```

do-while

```

int i = 1;

do {

```

```
    cout << i;

    i++;
} while (i <= 5);
```

11. How are break and continue statements used in loops? Provide examples.

❖ break and continue Statements (With Examples)

break

Used to **exit a loop** immediately.

Example:

```
for (int i = 1; i <= 10; i++) {

    if (i == 5)

        break;        // loop stops at 4

    cout << i << " ";

}
```

continue

Skips the current iteration and moves to the **next** one.

Example:

```
for (int i = 1; i <= 5; i++) {

    if (i == 3)

        continue;    // skips printing 3

    cout << i << " ";

}
```


12. Explain nested control structures with an example.

❖ Nested Control Structures (With Example)

Definition

Using one control structure (loop or condition) **inside another**.

Examples:

Loop inside a loop

if inside for

switch inside while

Example: Nested for Loop

Print a 3×3 pattern:

```
for (int i = 1; i <= 3; i++) {  
    for (int j = 1; j <= 3; j++) {  
        cout << "* ";  
    }  
    cout << endl;  
}
```

Output:

```
* * *  
  
* * *  
  
* * *
```

13. What is a function in C++? Explain the concept of function declaration, definition, and calling.

Function

A function is a block of code that performs a specific task.
It helps in **code reuse, modularity, and readability**.

Function Declaration (Prototype)

Tells the compiler **about the function** before it is used.

```
int add(int, int) ;
```

Function Definition

Contains the **actual body** of the function.

```
int add(int a, int b) {  
    return a + b;  
}
```

Function Calling

Invoking the function to execute its code.

```
int result = add(5, 3) ;
```

14. What is the scope of variables in C++? Differentiate between local and global scope.

Scope of Variables (Local vs Global)

Scope

Scope refers to **where a variable can be accessed** in the program.

Local Variables

Declared **inside** a function or block

Accessible only **within that block**

Destroyed after the block ends

Example:

```
void test() {  
    int x = 10;    // local variable  
}
```

Global Variables

Declared **outside all functions**

Accessible from **any function** in the program

Example:

```
int x = 100;    // global variable  
  
void show() {  
    cout << x;  
}
```

15. Explain recursion in C++ with an example.

Recursion

Recursion is a process where a function **calls itself** to solve a problem.

Example: Factorial using Recursion

```
int factorial(int n) {  
    if (n == 1)  
        return 1;
```

```
        return n * factorial(n - 1);  
    }
```

Call:

```
cout << factorial(5);
```

Works like:

```
5 * 4 * 3 * 2 * 1
```

16. What are function prototypes in C++? Why are they used?

Function Prototype

A function prototype is the **declaration** of a function before it is used.

```
float area(float, float);
```

Purpose of Function Prototypes

Tell the compiler **about the function** (name, return type, parameters)

Allow functions to be called **before their definition**

Helps detect **errors** in parameter types and number

17. What are arrays in C++? Explain the difference between single-dimensional and multi-dimensional arrays.

Array

An array is a collection of **multiple values of the same data type** stored in **continuous memory locations**.

Example:

```
int marks[5];    // array of 5 integers
```

Single-Dimensional Array (1D)

Stores data in **a single row**.

Accessed using **one index**.

Example:

```
int a[5] = {10, 20, 30, 40, 50};
```

```
cout << a[2];    // Output: 30
```

Multi-Dimensional Array (2D, 3D...)

Stores data in **rows and columns** (like a table).

Accessed using **multiple indexes**.

2D Example (matrix):

```
int m[2][3] = {
```

```
    {1, 2, 3},
```

```
    {4, 5, 6}
```

```
};
```

```
cout << m[1][2];    // Output: 6
```

18. Explain string handling in C++ with examples.

1. C-style Strings (char arrays)

Example:

```
char name[10] = "Sahil";
```

```
cout << name;
```

Functions used (from <cstring>):

```
strlen (name) ;
```

```
strcpy (name, "Hello") ;
```

```
strcmp (name, "World") ;
```

❏ 2. C++ Strings (string class)

(Recommended, easier to use)

Example:

```
#include <string>
```

```
string name = "Sahil";
```

```
cout << name;
```

Useful functions:

```
name.length() ;
```

```
name.append (" Kumar") ;
```

```
name.substr (0, 3) ;
```

19. How are arrays initialized in C++? Provide examples of both 1D and 2D arrays

Array Initialization in C++ (1D & 2D)

1. Initializing a 1D Array

Method 1: Full Initialization

```
int a[5] = {1, 2, 3, 4, 5};
```

Method 2: Partial Initialization

```
int a[5] = {10, 20};    // remaining become 0
```

Method 3: Compiler counts size

```
int a[] = {5, 6, 7};
```

2. Initializing a 2D Array

Method 1: Row-wise initialization

```
int m[2][3] = {  
    {1, 2, 3},  
    {4, 5, 6}  
};
```

Method 2: In one line

```
int m[2][3] = {1, 2, 3, 4, 5, 6};
```

Access Example:

```
cout << m[0][1];    // Output: 2
```

20. Explain string operations and functions in C++

In C++, strings can be handled in two ways:

C-style strings (character arrays)

std::string class from the <string> library (most commonly used)

common String Operations (using **std::string**)

1. String Initialization

```
string s1 = "Hello";  
string s2("World") ;
```

2. Concatenation (Joining strings)

```
string a = "Hello ";  
string b = "World";  
string c = a + b;           // c = "Hello World"
```

3. Getting String Length

```
string s = "Programming";  
cout << s.length();    // 11
```

4. Accessing Characters

```
string s = "Hello";  
cout << s[1];    //'e'
```

5. Taking Input

Using `cin` (stops at space):

```
string name;  
cin >> name;
```

Using `getline` (accepts spaces):

```
string sentence;  
getline(cin, sentence);
```

6. Comparing Strings

```
string a = "Apple";
```



```
string b = "Banana";
```

```
if (a == b) { }
```

7. Substring (extract part of string)

```
string s = "HelloWorld";
```

```
string sub = s.substr(0, 5); // "Hello"
```

8. Insert, Erase, Replace

```
string s = "HelloWorld";
```

```
s.insert(5, " "); // "Hello World"
```

```
s.erase(5, 1); // removes 1 char at index 5
```

```
s.replace(5, 5, " C++"); // "Hello C++"
```

9. Find a word or character

```
string s = "Hello World";
```

```
int pos = s.find("World"); // returns 6
```

Common String Functions (C-style strings) — using `<cstring>`

Function	Purpose	Example
----------	---------	---------

<code>strlen(str)</code>	Length	<code>strlen("Hello") = 5</code>
--------------------------	--------	----------------------------------

<code>strcpy(a,b)</code>	Copy string	<code>strcpy(a, b)</code>
--------------------------	-------------	---------------------------

<code>strcat(a,b)</code>	Concatenate	<code>strcat(a, "Hi")</code>
--------------------------	-------------	------------------------------

Function	Purpose	Example
<code>strcmp(a,b)</code>	Compare	<code>strcmp("A","B")</code>

Example:

```
char s1[20] = "Hello";
```

```
char s2[20] = "World";
```

```
strcat(s1, s2); // "HelloWorld"
```

21. Explain the key concepts of Object-Oriented Programming (OOP).

OOP (Object-Oriented Programming) is a programming approach that uses **objects** and **classes** to structure programs.

The four main concepts are:

1. Encapsulation

Wrapping data and functions into a single unit (class).

Data hiding is done using access specifiers (private, protected, public).

2. Inheritance

One class (child) can acquire properties of another class (parent).

Promotes code reusability.

3. Polymorphism

One name, many forms.

Types:

Compile-time (function overloading, operator overloading)

Run-time (function overriding using virtual functions)

4. Abstraction

Shows only important details.

Hides complex implementations from the user.

22. What are classes and objects in C++? Provide an example.

Class

A class is a **user-defined data type**.

It contains **data members** and **member functions**.

Object

An object is an **instance** of a class.

It is a real-world entity that uses class members.

Example:

```
#include <iostream>
```

```
using namespace std;
```

```
class Student {
```

```
public:
```

```
    string name;
```

```
    int age;
```

```
    void display() {
```

```
        cout << name << " " << age;
```

```
    }
```

```
};
```

```

int main() {

    Student s1;           // object

    s1.name = "Rohan";

    s1.age = 20;

    s1.display();

}

```

23. What is inheritance in C++? Explain with an example.

Inheritance allows a **child class (derived class)** to acquire properties and functions of a **parent class (base class)**.

Example:

```
#include <iostream>
```

```
using namespace std;
```

```
class Animal {           // Base class
```

```
public:
```

```
    void eat() {
```

```
        cout << "Eating...";
```

```
    }
```

```
};
```

```
class Dog : public Animal { // Derived class
```

```
public:
```

```

    void bark() {
        cout << "Barking...";
    }
};

int main() {
    Dog d;

    d.eat(); // inherited
    d.bark(); // own function
}

```

Output:

```

Eating...
Barking...

```

24.What is encapsulation in C++? How is it achieved in classes?

Encapsulation

Encapsulation means **binding data and functions into a single unit** (class) and **restricting direct access** to the data.

How is it achieved?

Using **access specifiers**:

`private` → data hidden

`public` → functions to access data

Example:

```
class BankAccount {
```

```
private:
```

```
    int balance;        // hidden data
```

```
public:
```

```
    void setBalance(int b) {
```

```
        balance = b;    // controlled access
```

```
    }
```

```
    int getBalance() {
```

```
        return balance;
```

```
    }
```

```
};
```